ROCsim - Team 1

Jiwon Lee | 251342617 | jlee3859@uwo.ca

Jacob Lee | 251362845 | jlee3887@uwo.ca

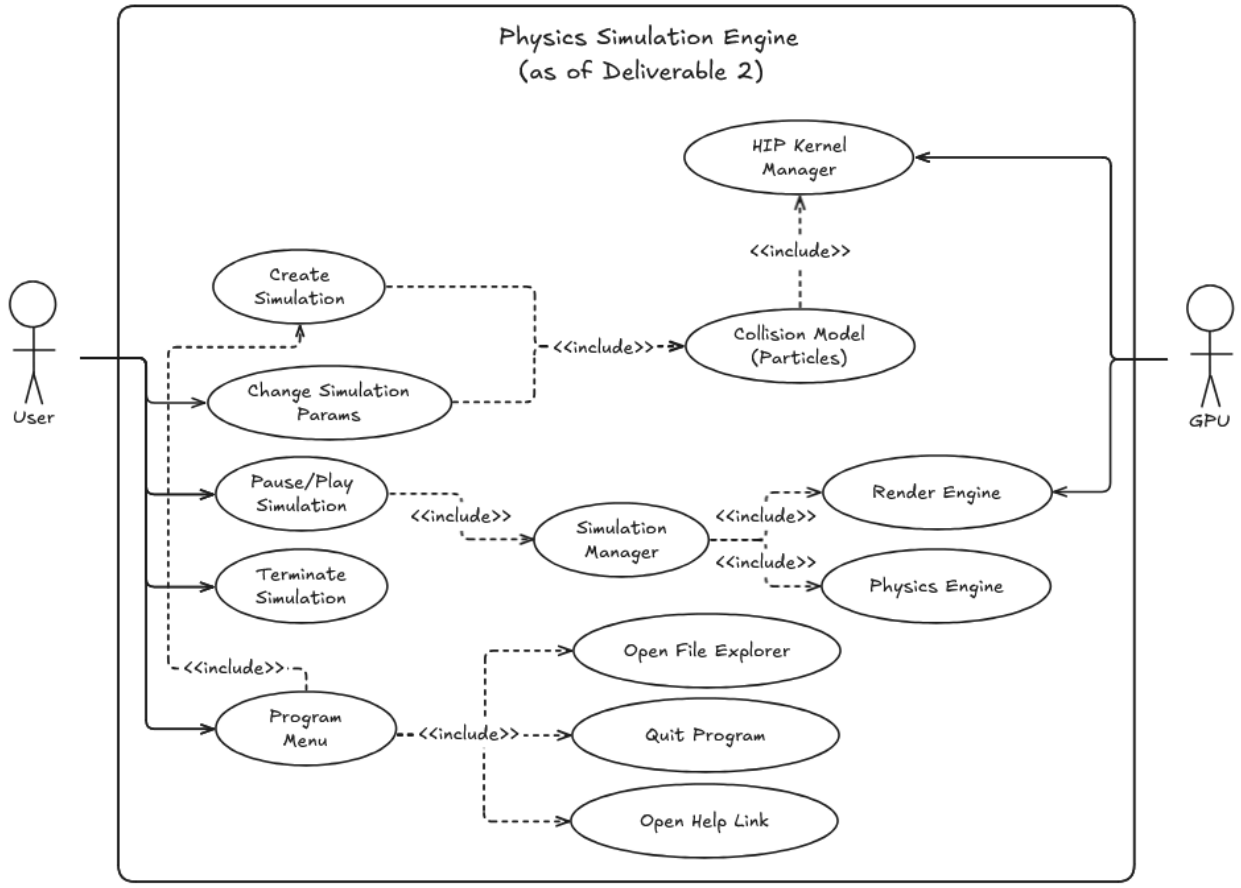
CS3307 - Deliverable 2 - UML & ` Rationale

November 9, 2025

1. Abstract

As stated, the following project proposal is a massive undertaking, with the approval of the professor, we split the responsibilities into two teams (with some overlapping tasks and documentations). Team 1 will focus on the low level GPU optimization while Team 2 focuses on the graphical aspect. For the logical side of it both Teams would need to collaborate to integrate each feature properly.
Header sections will be marked with Header* to denote that the group specific information will be highlighted in **YELLOW**, i.e., that information will only be seen on the relevant team's document. Should a piece of information not be highlighted (while the header has a *), then that information is the **SAME** for both teams, i.e., the information is intercorrelated so it is repeated in both documents.

2. Use Case Diagram



2.1 Actors

User (Primary Actor): Able to use the physics engine by simulating particles, change simulation parameters: toggle collisions, grids, change size, enable HIP plugin, toggle downward gravity, modify damping & simulation speed.

GPU (Secondary Actor): Assists by offloading complex physics calculations to the HIP Kernel Manager. (GPU processes these queries in parallel to speed up simulation). It also interacts with the rendering engine by helping render visual representations of the simulation data.

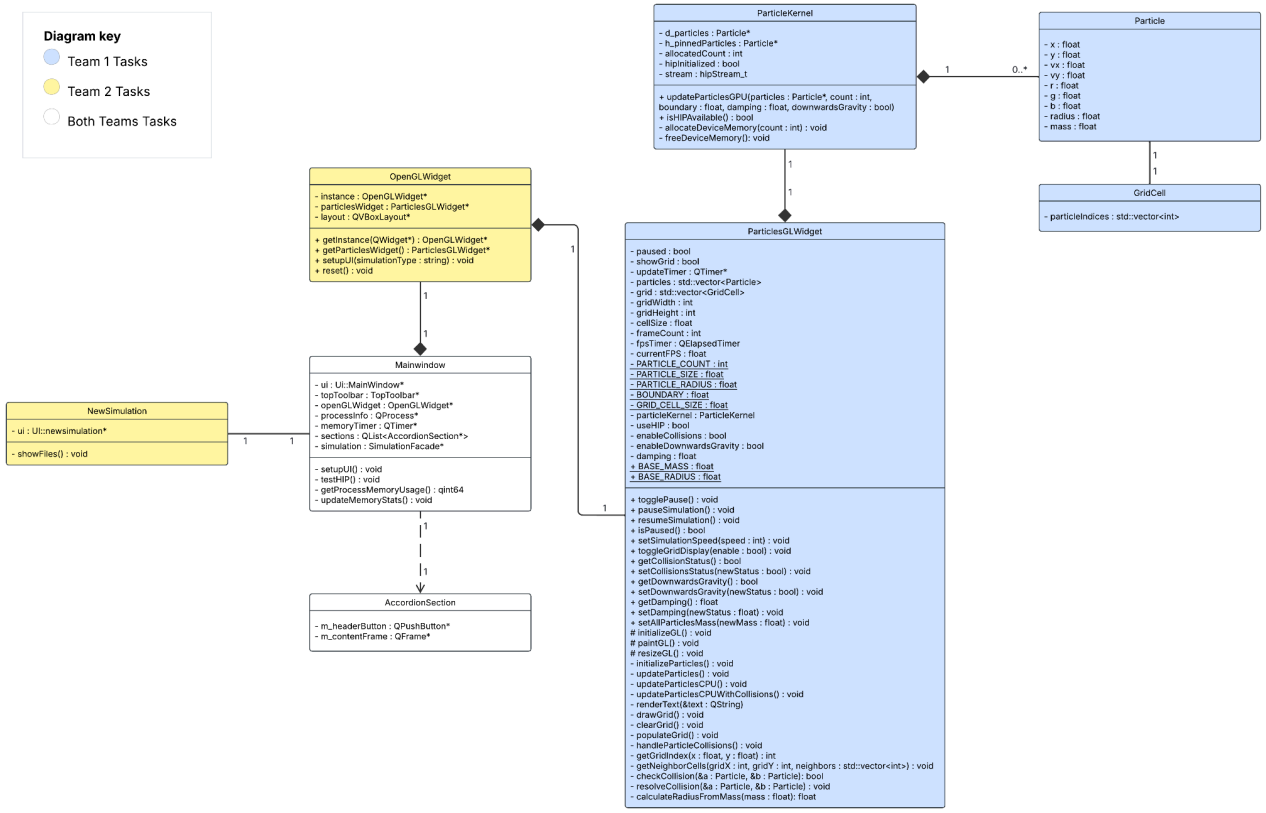
2.2 Use Cases

Use Case: Create Simulation ○ Actors: User ○ Description: Users can create a new simulation environment where they can select (or modify) physics models. ○ Pre-conditions: System is initialized and ready to create a new simulation via the program menu. ○ Post-conditions: User has modified the physics model for the simulation.	**Use Case: Program Menu** ○ Actors: User ○ Description: Users can use shortcuts or the program menu to exit the program, create new simulation, open help, etc... ○ Pre-conditions: Program has been open ○ Post-conditions: User has interacted with the menu, giving them the desired outcome (easy to navigate)
Use Case: Change Simulation Params ○ Actors: User ○ Description: Users can change object (particle) size and shape, toggle on and off collisions, grids, gravity, and HIP plugin. They can also adjust the particle damping value and simulation speed. ○ Pre-conditions: Users have opened the application. ○ Post-conditions: User modifies the simulation.	**Use Case: Open File Explorer** ○ Actors: User ○ Description: The program opens the user's file explorer to import an old simulation. (currently it only opens the file explorer and the simulation doesn't read JSON files yet) ○ Pre-conditions: Users have to click the 'Open Simulation' button found in the file menu bar. ○ Post-conditions: User's file explorer is opened to import an old simulation they saved onto their computer. (currently only opens the file explorer)
Use Case: Pause/Play Simulation ○ Actors: User ○ Description: Users can play or pause the current simulation. ○ Pre-conditions: Users have to press the play button or pause button. ○ Post-conditions: The simulation is played or paused.	**Use Case: Quit Program** ○ Actors: User ○ Description: Users can exit the program ○ Pre-conditions: User goes to program menu and clicks 'Exit' or use the shortcut CTRL+Q ○ Post-conditions: Program exits

Use Case: Stop Simulation ○ Actors: User ○ Description: Users can stop the running simulation at any time. ○ Pre-conditions: The simulation is currently running. ○ Post-conditions: The simulation is stopped and the environment is reset.	**Use Case: Open Help Link** ○ Actors: User ○ Description: Users can see ROCsim' documentation ○ Pre-conditions: User goes to program menu and clicks 'About ROCsim' ○ Post-conditions: Users gets redirected to the GitHub page of the ROCsim repo
Use Case: Simulation Manager ○ Actors: None ○ Description: Ties the graphics (render engine) and logical (physics engine) to create the physics simulation. ○ Pre-conditions: Render and Physics engines are working. ○ Post-conditions: Displays the physic simulation accurately.	**Use Case: Render Engine** ○ Actors: GPU ○ Description: Generates visual of the simulation environment, rendering objects. ○ Pre-conditions: The GPU is available and initialized for rendering tasks. ○ Post-conditions: The rendered scene is updated in real time or per frame based on the simulation state.
Use Case: Physics Engine ○ Actors: None ○ Description: The physics engine calculates and simulates the physical interactions and behaviors within the simulation. ○ Pre-conditions: A physics model is selected and the simulation is running. ○ Post-conditions: The simulation state is updated based on physical interactions.	**Use Case: HIP Kernel Manager** ○ Actors: GPU ○ Description: Controls the execution and optimization, offloading computational tasks to the GPU to improve efficiency. ○ Pre-conditions: The GPU is initialized and available for computation. ○ Post-conditions: The GPU is actively running or optimizing the kernel.
Use Case: Collision Model (particles) ○ Actors: None ○ Description: Defines the rules and equations that govern the physical behavior in the simulation. ○ Pre-conditions: A valid physics model is selected or created for the simulation. ○ Post-conditions: The simulation runs based on the chosen physics model.	

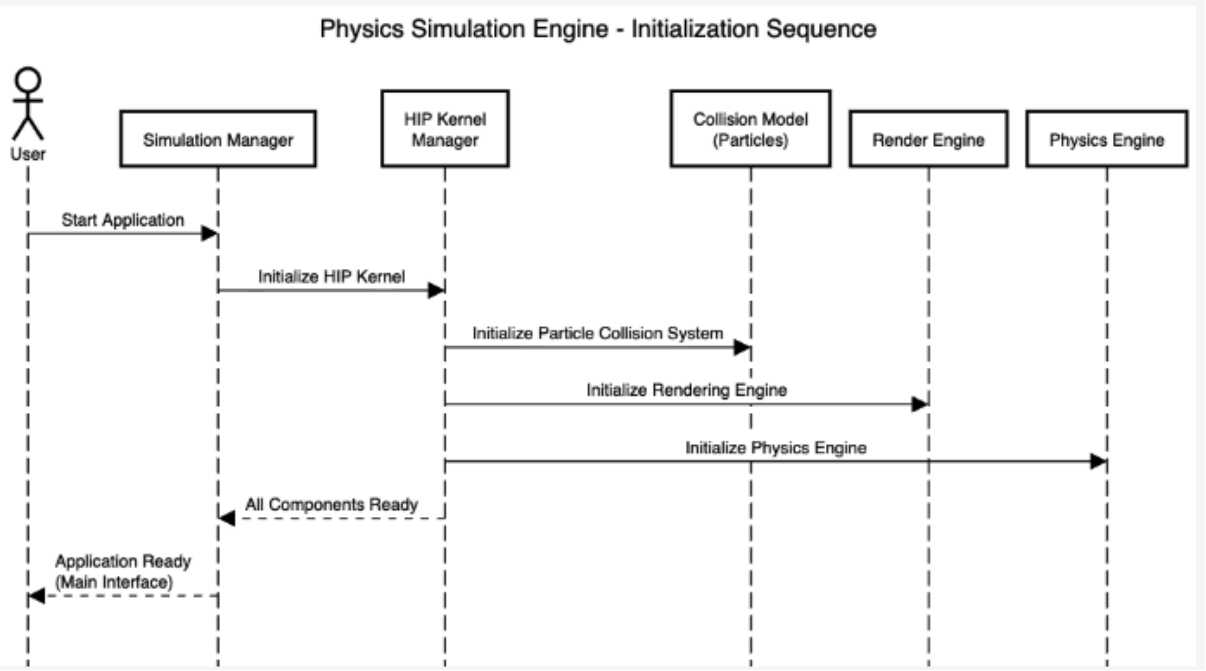
3. Class Diagram*

Note: Constructors and Destructors have been omitted for clarity because including them would increase visual density of the diagrams (but they do exist in our code) and `private slots` and `signals` are also not included (as it's Qt auto generated and not inherently intentionally designed by us).

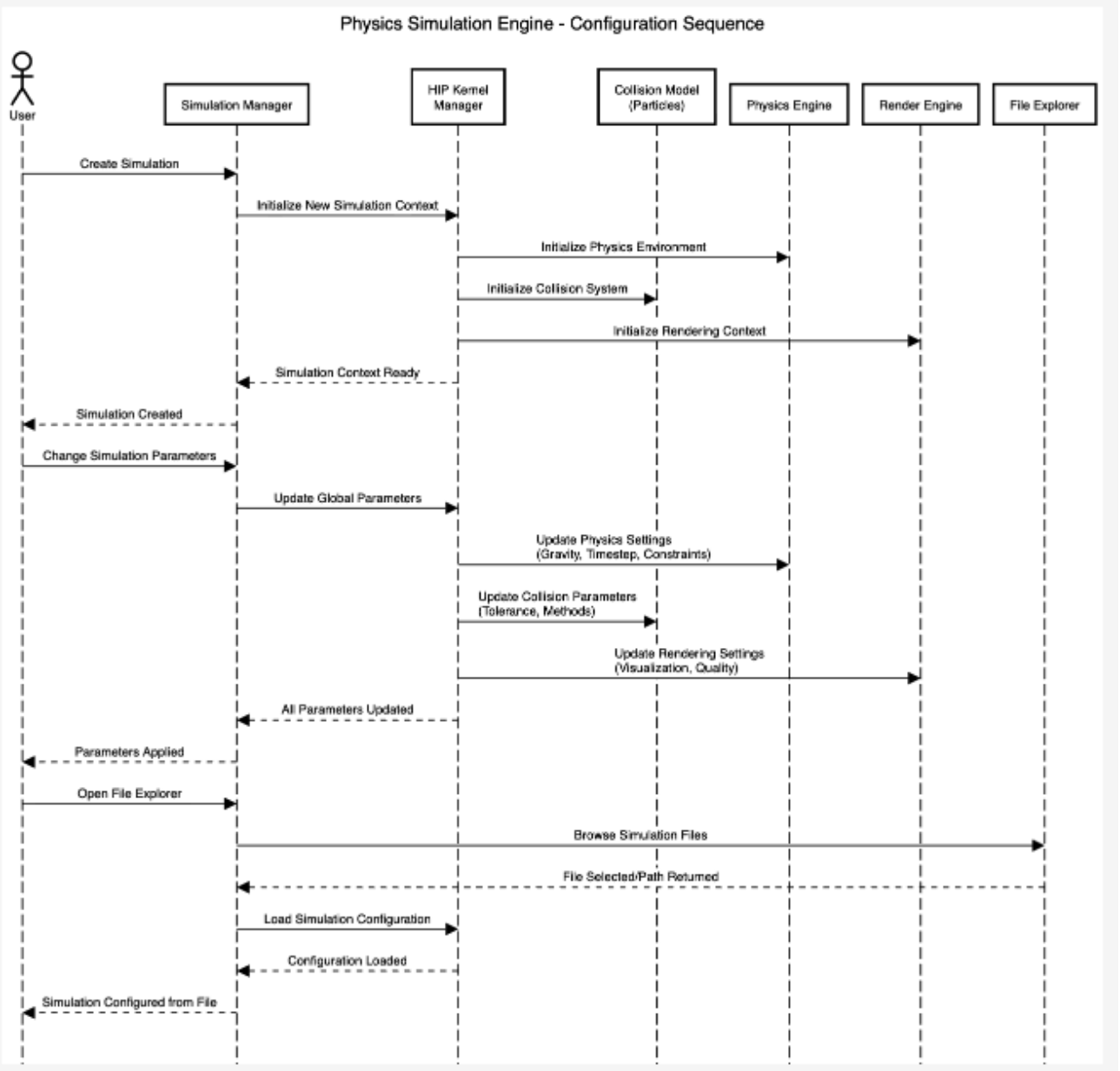


4. Sequence Diagram

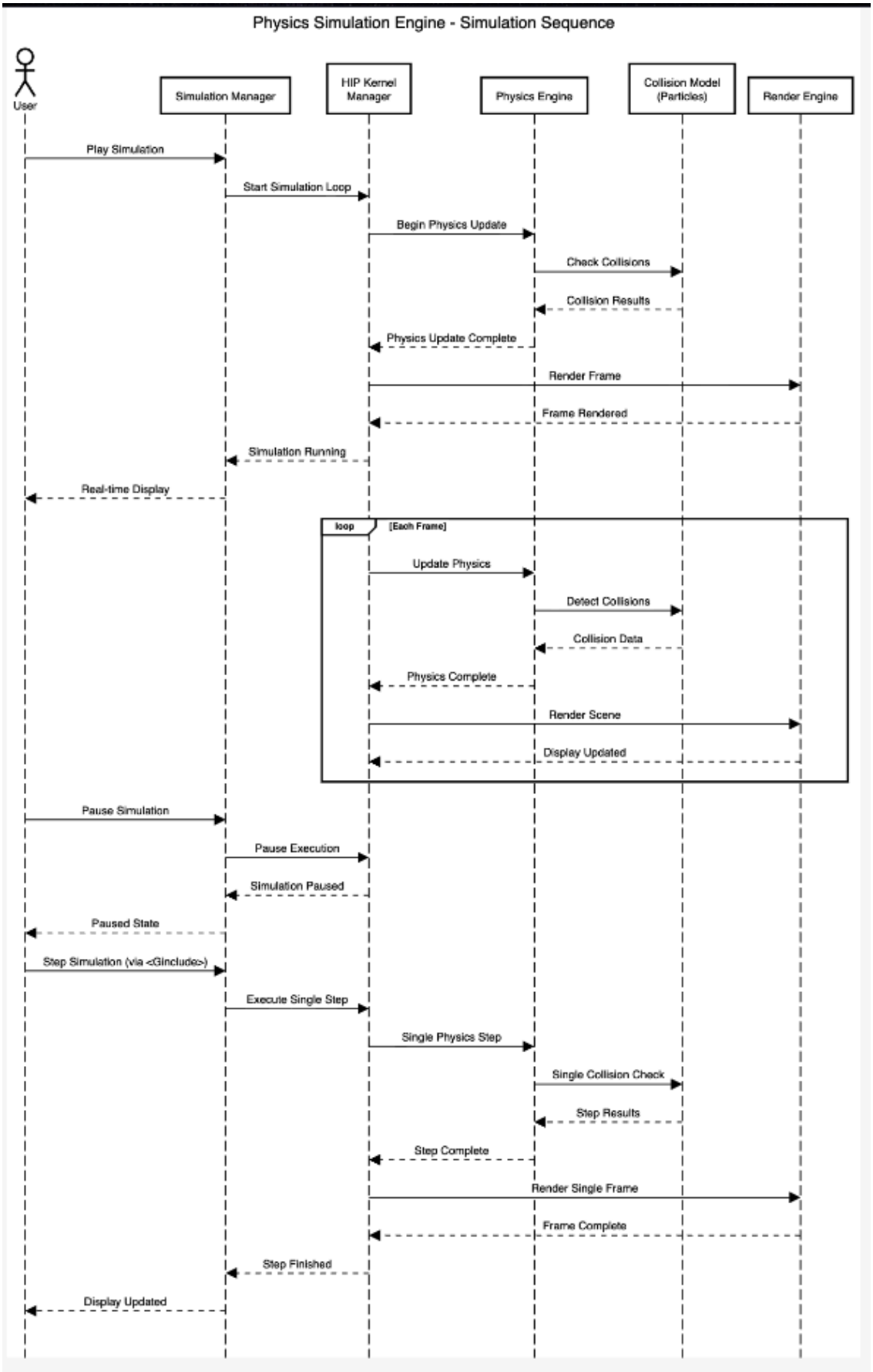
4.1 Initialization Sequence



4.2 Configuration Sequence



4.2 Simulation Sequence



5. Design Rationale*

Design Decisions/Patterns

Singleton Pattern - Creational

Implementation: `OpenGLWidget` class

Justification: It ensures only one instance of the OpenGL rendering context exists throughout the application lifecycle:

- OpenGL contexts creation is resource-intensive and having multiple contexts leads to significant memory and performance
- Only one is needed because multiple instances would conflict over GPU resources
- Prevents memory leaks from duplicate widget creation

The static `getInstance()` method controls instantiation, and the copy constructor/assignment operator is deleted to prevent duplication.

Strategy Pattern (Implicit) - Behavioural

Implementation: Particle update methods in `ParticlesGLWidget`

Justification: Allows different algorithms to run on particle physics depending on the following runtime conditions. The system will automatically select the appropriate strategy based on the user's hardware:

- `updateParticlesGPU()` - HIP/ROCm GPU acceleration strategy
- `updateParticlesCPU()` - Basic CPU physics without collisions
- `updateParticlesCPUWithCollisions()` - CPU with spatial grid collision detection

Additional Architectural Decisions

Spatial Grid Collision Detection:

Implemented O(n) collision detection using uniform spatial grid partitioning instead of

naive O(n²) approach, where particles are hashed into grid cells, and collision checks only occur between particles in neighboring cells

VBO-Based Rendering:
Replaced immediate mode OpenGL with modern Vertex Buffer Objects (VBOs) and shader-based rendering for 10-100x performance improvement. Particle data is uploaded to the GPU once per frame using `glBufferSubData()`

Hybrid CPU/GPU Architecture:
Conditional compilation with `#ifdef HIP_ENABLED` allows the application to run on systems without AMD GPUs while maintaining high performance on supported hardware

Changes from Deliverable 1

Architecture Refinements

- Refined OpenGLWidget responsibilities - We originally planned to handle both rendering and simulation; split into OpenGLWidget (container) and ParticlesGLWidget (simulation engine)

Design Pattern Additions

- Strategy pattern emergence - Not explicitly planned but emerged naturally from CPU/GPU branching logic

Feature Changes

- Removed planned features:
  - Audio feedback (`QMediaPlayer` included but not actively used)
- Added features:
  - Real-time FPS and memory monitoring
  - Damping coefficient slider
  - Simulation speed control (5x slower to 5x faster)
  - Mass/size adjustment with live updates

Technology Stack Changes

- OpenGL modernization - Upgraded from immediate mode to shader-based VBO rendering
- CMake improvements - Conditional HIP compilation, better platform detection

Physics Engine Enhancements

- Collision detection - Implemented spatial grid algorithm, not detailed in Deliverable 1
- Variable damping - Added configurable energy loss on collisions
- Gravity simulation - Optional downward force application
- Mass-radius coupling - Particle visual size scales with mass

Next Steps

Immediate Priorities

- Complete HIP-OpenGL Interoperability - Currently using `glBufferSubData()` for CPU-to-GPU transfer and zero-copy rendering with `hipGraphicsGLRegisterBuffer()` will be implemented
- Implement particle-particle collisions on GPU - Currently only available in CPU mode. It will have a port spatial grid algorithm that goes to HIP kernel
- Simulation Freezing - Simulation seems to freeze after every few frames for a few milliseconds. Optimizing the QTimer will be tested to see if this problem is resolved

Performance Optimizations

- Instanced rendering - GPU utilization will be improved by rendering all particles in single draw call with instancing
- Asynchronous GPU updates - HIP streams will be used to overlap computation and rendering
- Adaptive time-stepping - Variable physics timestep will be implemented based on particle density

Advanced Features

- Force fields - Add attractors/repellers users can place in scene
- Particle emitters - Dynamic particle creation/destruction
- Boundary shapes - Support for non-rectangular simulation bounds

New/Incoming Features

- Other simulation types - Springs networks, inter-object gravity
- NVIDIA GPU Support - HIP Configuration for NVIDIA GPUs
- Further customizability - Changing of number of particles, multiple types of particles, fine-tuned control over each type of object(s)

Testing & Documentation

- Unit tests - Add Google Test framework for physics calculations
- Performance benchmarks - Automated FPS testing with varying particle counts

Cross-Platform Support

- Linux Testing - Currently is WSL2-dependent for HIP support. We are still planning on adding native Windows HIP support and for the application to run on native Linux Ubuntu
- Docker deployment - Finalize containerized distribution for Linux systems since HIP is still having trouble running through a Linux Docker container

6. Video Walkthrough

Considering that this project is being worked on by 2 groups, instead of having a normal range of 3-5 min for the video, we doubled the time. This was a necessity due to the sheer complexity of our project. It should be approximately ~9 min. Thus, falling within the (2x) 6-10 min range.

Video Link: 3307 Deliverable 2 (video walkthrough).mp4